

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Колледж программирования и кибербезопасности

УЧЕБНАЯ ДИСЦИПЛИНА/
МЕЖДИСЦИПЛИНАРНЫЙ КУРС
МДК.01.04 Системное программирование

ПЦК Программирование

ВИД УЧЕБНОГО
МАТЕРИАЛА Лекция - 2

ПРЕПОДАВАТЕЛЬ Мухортова Наталья Николаевна

Основы программирования на языке Си.

Указатели

Файловый ввод-вывод.

Введение

Указатель — это переменная адресного типа, которая содержит адрес некоторого элемента данных (переменной, константы, функции, структуры). Указатель, как и другие переменные, имеет тип данных и идентификатор. Однако указатели используются таким образом, который отличается от обычных переменных.

Ключевые принципы указателей:

1. Указатель — это переменная, в которой хранится адрес.
2. Утверждение вида «A указывает на B» означает, что «A содержит адрес B».

Каждая переменная в программе — это объект, имеющий имя и значение. Имя переменной соответствует адресу выделенного для неё участка памяти, а значение — содержимому этого участка. Адрес переменной можно получить с помощью унарной операции &, а значение по адресу — с помощью *.

Пример:

```
int a = 5;
int *p = &a;  // p хранит адрес переменной a
printf("%d", *p);  // разыменование указателя возвращает значение 5
```

Операции над указателями

В языке Си с указателями допустимы следующие операции:

1. **Присваивание**: указателю можно присвоить адрес.
2. **Разыменование**: операция `*` получает значение по адресу.
3. **Получение адреса указателя**: операция `&` возвращает адрес переменной.
4. **Арифметика указателей**: добавление или вычитание целых чисел изменяет адрес, хранящийся в указателе.
5. **Сравнение указателей**: например, `==`, `!=`, `<`, `>` и другие.

Пример:

```
int arr[] = {10, 20, 30};
int *p = arr;
p++;  // теперь p указывает на второй элемент массива
printf("%d", *p);  // вывод: 20
```

Нетипизированный указатель (void *)

`void *` — это указатель, который может хранить адрес любого типа данных, но разыменование такого указателя напрямую невозможно.

Пример:

```
void *ptr;
int a = 10;
ptr = &a;
int *p = (int *)ptr;
printf("%d", *p);  // вывод: 10
```

Указатели и const

Указатели могут быть:

1. `const int *p` — нельзя изменить значение, на которое указывает `p`.
2. `int *const p` — нельзя изменить сам указатель `p`.
3. `const int *const p` — ни значение, ни указатель менять нельзя.

Пример:

```
const int a = 10;
const int *p = &a;  // нельзя изменить значение через *p
```

Указатель на указатель

Указатель может указывать на другой указатель. Это называется **указатель на указатель** и используется для работы с многомерными массивами и сложными структурами данных.

Пример:

```
int a = 10;
int *p = &a;
int **pp = &p;
printf("%d", **pp);  // вывод: 10
```

Указатель файла

Указатель файла связывает систему ввода-вывода языка Си. Для работы с файлами подключается библиотека `<stdio.h>`.

Пример:

```
FILE *file_ptr = fopen("data.txt", "r");
if (file_ptr != NULL) {
    // работа с файлом
    fclose(file_ptr);
}
```

Указатели и массивы

Имя массива — это константный указатель на его первый элемент. Элементы массива можно обрабатывать через арифметику указателей:

Пример:

```
int arr[] = {10, 20, 30};
int *p = arr;
printf("%d", *(p + 2));  // вывод: 30
```

Передача массива в функцию может осуществляться через указатель:

```
void printArray(int *arr, int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", *(arr + i));
    }
}
```

Указатели на функции

Указатели могут хранить адреса функций. Это позволяет передавать функции как параметры.

Пример:

```
int add(int a, int b) { return a + b; }
int (*func_ptr)(int, int) = add;
printf("%d", func_ptr(3, 4));  // вывод: 7
```

Выводы

- **Указатель** — это переменная, хранящая адрес другой переменной.
- Операции `*` (разыменование) и `&` (взятие адреса) являются основными для работы с указателями.
- Указатели играют важную роль в работе с массивами, функциями и динамическим выделением памяти.
- Работа с указателями требует аккуратности, так как ошибки могут привести к сбоям программы.
-

Введение в динамическую память

Динамическая память выделяется во время выполнения программы, что позволяет гибко управлять объемом памяти в зависимости от потребностей. В отличие от статической памяти, размер которой фиксирован на этапе компиляции, динамическая память предоставляет больше свободы в работе с большими и изменяемыми объемами данных.

В языке C динамическая память управляется вручную, что требует от программиста внимания и аккуратности. Ошибки, такие как утечка памяти или использование невалидных указателей, могут привести к сбоям программы.

Преимущества динамической памяти

- Позволяет создавать структуры данных, размер которых неизвестен заранее (например, массивы переменной длины).
- Упрощает управление памятью для сложных программ.
- Динамическая память позволяет эффективно использовать ресурсы, выделяя память только по мере необходимости.

Структура памяти программы

Память программы делится на несколько сегментов:

- **Стек**: используется для хранения локальных переменных и аргументов функций. Работает по принципу LIFO (Last In, First Out).
- **Куча**: область динамической памяти, в которой память выделяется и освобождается вручную с помощью функций `malloc`, `calloc`, `realloc` и `free`.
- **Сегмент данных**: хранит глобальные переменные и статические данные программы.
- **Сегмент кода**: содержит исполняемый код программы.

Каждый сегмент выполняет свою роль в обеспечении корректной работы программы, а управление памятью в куче — одна из ключевых задач программиста.

Функции для динамического распределения памяти

Для управления динамической памятью в языке C используется стандартная библиотека `<stdlib.h>`, которая предоставляет следующие функции:

- **`malloc(size_t size)`**: выделяет блок памяти указанного размера.
- **`calloc(size_t n, size_t size)`**: выделяет память для массива из `n` элементов и инициализирует их нулями.
- **`realloc(void *ptr, size_t size)`**: изменяет размер ранее выделенного блока памяти. Если новый размер меньше, лишняя память освобождается. Если новый размер больше, выделяется дополнительная память.
- **`free(void *ptr)`**: освобождает ранее выделенную память.

Пример использования функций:

```
#include <stdlib.h>
#include <stdio.h>

int main() {
    int *arr = (int *)malloc(5 * sizeof(int)); // Выделение памяти для 5 элементов
    if (arr == NULL) {
        printf("Ошибка выделения памяти\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        arr[i] = i * i; // Заполнение массива
        printf("%d ", arr[i]);
    }
    printf("\n");

    free(arr); // Освобождение памяти
    return 0;
}
```

Основные правила работы с динамической памятью

1. Всегда проверяйте, что результат вызова `malloc` или `calloc` не равен `NULL`. Это гарантирует, что память была успешно выделена.
2. После завершения работы с памятью обязательно вызывайте `free`, чтобы избежать утечек.
3. Никогда не используйте указатель после вызова `free` — это приведет к неопределенному поведению программы.
4. В случае использования `realloc` всегда проверяйте результат, поскольку эта функция может вернуть `NULL` в случае неудачи, а старый указатель может быть всё еще действительным.

Пример с проверкой выделения памяти:

```
int *ptr = (int *)malloc(10 * sizeof(int));
if (!ptr) {
   printf("Ошибка: не удалось выделить память\n");
   exit(EXIT_FAILURE);
}
free(ptr);
```

Одномерные динамические массивы

Одномерный динамический массив — это массив, размер которого определяется во время выполнения программы. Для создания такого массива в C используется функция malloc.

Пример создания массива:

```
#include <stdlib.h>
#include <stdio.h>
```

```
int main() {
    int n;
    printf("Введите размер массива: ");
    scanf("%d", &n);

    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) {
        printf("Ошибка выделения памяти\n");
        return 1;
    }

    for (int i = 0; i < n; i++) {
        arr[i] = i + 1;
        printf("%d ", arr[i]);
    }
    printf("\n");

    free(arr);
    return 0;
}
```

> Если размер массива нужно изменить, можно использовать функцию `realloc`, которая перераспределяет память.

Динамическое выделение памяти для двумерных массивов

Для двумерных массивов память можно выделять двумя способами:

1. Одним блоком (плоский массив)

```
int *matrix = (int *)malloc(rows * cols * sizeof(int));
```

Доступ к элементу: `matrix[i * cols + j]`.

2. Массив указателей

```
#include <stdlib.h>
#include <stdio.h>

int main() {
    int rows, cols;
    printf("Введите количество строк и столбцов: ");
    scanf("%d %d", &rows, &cols);

    int **matrix = (int **)malloc(rows * sizeof(int *));
    for (int i = 0; i < rows; i++) {
        matrix[i] = (int *)malloc(cols * sizeof(int));
    }

    // Заполнение и вывод
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = i + j;
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }

    // Освобождение памяти
    for (int i = 0; i < rows; i++) {
        free(matrix[i]);
    }
    free(matrix);

    return 0;
}
```

Важные моменты

- Для двумерных массивов всегда следите за тем, чтобы каждая строка имела выделенную память, если вы используете массив указателей.
- Важно освобождать память в том же порядке, в каком она была выделена.

Заключение

Работа с динамической памятью открывает широкие возможности для управления памятью в программах, но требует осторожности. Ошибки, такие как утечки памяти, двойное освобождение памяти или использование невалидных указателей, могут привести к сбоям программы. В современных языках программирования, таких как Python или Java, управление памятью происходит автоматически, но в C и C++ программисты несут полную ответственность за правильное распределение и освобождение памяти. Это требует дисциплины и внимания на каждом этапе работы с динамической памятью.

Основы файловых систем

Что такое файл?

Файл — это именованная область данных, хранящаяся на носителе информации (жесткий диск, SSD, флешка и т.д.), доступная по уникальному имени. ОС могут рассматривать как файлы и обрабатывать сходным образом и другие ресурсы, такие как устройства (клавиатура, экран, принтер), потоки данных и сетевые ресурсы.

Файловая система (ФС)

Файловая система (ФС) — это:

- **Набор правил и структур**, описывающих, как данные (файлы и каталоги) организуются и хранятся на носителе.
- **Совокупность всех файлов**, хранимых в компьютерной системе.
- (В контексте UNIX-подобных систем) **Совокупность всех файлов на разделе диска или устройстве** вместе с самими устройствами, которые также представлены как файлы.

По сути, *ФС* определяет порядок и способ именования, хранения и организации информации. Практические реализации файловых систем, например, `NTFS` или `ext4`, — это технический способ организации информации на определенном типе носителя в соответствии с принятыми правилами.

Структура файловой системы Linux

В отличие от операционных систем семейства Windows, где каждый логический или физический раздел диска имеет свою букву и независимую иерархию (например, `C:\`, `D:\`), в Linux используется **единая иерархическая структура** с одним **корневым каталогом (`/`)**. Все устройства монтируются в единое дерево каталогов.

Ключевые особенности структуры ФС Linux

- **Единый корень (`/`)**: Вся файловая система начинается с корневого каталога.
- **Монтирование устройств**: Дополнительные разделы диска, USB-флешки, оптические диски и другие носители подключаются (монтируются) к определенным каталогам внутри этого единого дерева, называемым **точками монтирования** (например, `/media` для съемных носителей или пользовательские точки монтирования). Это делает пути к данным более стабильными, независимо от того, на каком физическом устройстве они расположены.
- **"Все есть файл"**: В Linux все, включая дисковые накопители и устройства (клавиатура, экран, принтер и т.д.), представлено в виде

файлов. Это ключевое понятие, позволяющее единообразно работать с различными ресурсами через файловые операции.
- **Иерархичность**: ФС Linux имеет единый корень (`/`).
- **Неизменность путей**: Пути к данным остаются постоянными даже при изменении физической структуры дисков благодаря монтированию.
- **Регистрозависимость**: Имена файлов и каталогов в Linux чувствительны к регистру символов (например, `Document.txt` и `document.txt` — это два разных файла).

Типы файлов в Linux (по первому символу в выводе команды `ls -l`)

В выводе команды `ls -l` первый символ строки показывает тип файла:

Символ	Тип файла	Описание
`-`	Обычный файл	Содержит данные (текст, программы, изображения и т.д.).
`d`	Каталог	Специальный файл, содержащий список других файлов и каталогов.
`l`	Символическая ссылка	Файл, указывающий на другой файл или каталог по имени.
`c`	Символьное устройство	Представляет устройства, работающие с потоками данных (например, терминал).
`b`	Блочное устройство	Представляет устройства, работающие с данными блоками (например, жесткий диск).
`p`	Именованный канал FIFO	Используется для межпроцессного взаимодействия (первым вошел - первым вышел).
`s`	Сокет	Используется для сетевого взаимодействия между процессами.

Основные каталоги в Linux

Вот краткое описание назначения некоторых важных стандартных каталогов корневой файловой системы Linux:

- `/`: Корень файловой системы.
- `/bin`: Основные исполняемые файлы (бинарники) пользовательских команд, необходимые для работы системы в однопользовательском режиме и доступные всем пользователям.
- `/sbin`: Системные исполняемые файлы, предназначенные для системного администрирования (часто доступны только суперпользователю root).
- `/etc`: Конфигурационные файлы системы и установленных программ.
- `/dev`: Файлы устройств.
- `/proc`: Виртуальная файловая система, содержащая информацию о запущенных процессах и ресурсах системы (создается ядром в оперативной памяти).
- `/sys`: Виртуальная файловая система, предоставляющая интерфейс к устройствам, подключенным к системе, и к ядру.
- `/var`: Переменные данные. Содержит файлы, которые часто изменяются в процессе работы системы (логи `/var/log`, кеш `/var/cache`, очереди печати и т.п.).
- `/tmp`: Временные файлы. Доступен для записи всем пользователям, содержимое часто удаляется при перезагрузке.
- `/usr`: Содержит основную часть программного обеспечения пользователя, библиотеки, документацию.
- `/home`: Домашние каталоги пользователей. В каждом подкаталоге (например, `/home/user1`) хранятся личные файлы и настройки конкретного пользователя.
- `/boot`: Файлы, необходимые для загрузки операционной системы (загрузчик, ядро).
- `/lib` / `/lib64`: Системные библиотеки, необходимые для работы программ из `/bin` и `/sbin`.
- `/opt`: Дополнительное программное обеспечение, устанавливаемое сторонними производителями.
- `/mnt`: Традиционное место для временного монтирования файловых систем (например, сетевых дисков, других разделов).
- `/media`: Точка монтирования для съемных носителей (USB-флешек, CD/DVD) при автоматическом монтировании в графической среде.

- `/srv`: Данные для сервисов, предоставляемых системой (например, файлы веб-сервера).
- `/run`: Временные файлы, содержащие информацию о запущенных процессах с момента последней загрузки системы.

Имена файлов и папок в Linux

Ограничения и особенности имен файлов и каталогов в Linux:

- Максимальная длина имени файла/каталога обычно составляет 255 символов.
- Символ `/` запрещен в именах, так как он используется как разделитель компонентов пути.
- Имя, начинающееся с точки (`.`), считается скрытым и не отображается по умолчанию командой `ls`.
- Имена чувствительны к регистру (`file.txt` ≠ `File.txt`).
- Пробелы в именах допускаются, но считаются плохим тоном в профессиональной среде. Использование пробелов и некоторых спецсимволов допустимо, но не рекомендуется для удобства работы в командной строке (требуют экранирования или заключения в кавычки) и для совместимости при копировании на другие файловые системы.

Что такое Inode?

Inode (индексный дескриптор) — это ключевая структура данных в файловых системах Linux (и других UNIX-подобных систем), которая действует как "паспорт" или "карточка" для каждого файла или каталога на диске.

Inode хранит всю **метаинформацию** об объекте файловой системы, кроме его имени: * Тип файла (обычный файл, каталог, ссылка и т.д.). * Права доступа (кто может читать, писать, выполнять). * Владельца и группу. * Размер файла. * Время создания, последнего доступа и модификации. * И самое главное — **указатели на блоки данных на диске**, где физически хранится содержимое файла.

Каждому Inode присваивается **уникальный номер** в рамках конкретной файловой системы.

Имя файла в каталоге — это просто **ссылка** или "ярлык", который связывает понятное человеку имя с соответствующим номером Inode. Когда вы обращаетесь к файлу по имени, система использует это имя, чтобы найти нужный Inode, а затем по информации из Inode находит сами данные файла на диске.

Несколько жестких ссылок могут указывать на один и тот же Inode, потому что они являются разными именами, ссылающимися на одну и ту же "карточку" с информацией об одних и тех же данных.

Управление файловыми системами на уровне ОС

- **/etc/fstab**: Этот конфигурационный файл содержит информацию о файловых системах и параметрах их автоматического монтирования (подключения) при загрузке системы. Каждая строка описывает одну файловую систему, указывая устройство (или его UUID), точку монтирования, тип ФС, опции монтирования (например, `defaults`, `auto`, `noauto`, `ro` - только чтение, `rw` - чтение-запись) и параметры проверки и резервного копирования.
- **fsck**: Утилита (`file system check`), используемая для проверки и восстановления целостности файловых систем на предмет ошибок и несоответствий. **Важно:** Применять `fsck` следует только к **размонтированным** файловым системам (за исключением корневой ФС, которую можно проверять в режиме только для чтения во время загрузки).

Основные команды Linux для работы с файлами

- `ls`: Вывести список файлов и каталогов. (`ls -l` - подробный список)
- `cd [каталог]`: Сменить текущий каталог.
- `pwd`: Показать полный путь к текущему каталогу.
- `mkdir [каталог]`: Создать новый каталог.
- `rm [файл/каталог]`: Удалить файл или пустой каталог. (`rm -r` - удалить каталог с содержимым).
- `mv [источник] [назначение]`: Переместить или переименовать файл/каталог.
- `cp [источник] [назначение]`: Скопировать файл/каталог.

- `cat [файл]`: Вывести содержимое файла на экран (или объединить несколько файлов).
- `grep [шаблон] [файл]`: Найти строки, соответствующие шаблону, в файле. (`grep -i` - игнорировать регистр).
- `ln [цель] [ссылка]`: Создать жесткую ссылку. (`ln -s [цель] [ссылка]` - создать символическую ссылку).
- `touch [файл]`: Создать пустой файл или обновить время последнего доступа/модификации.

Права доступа к файлам в Linux

Linux использует классическую систему прав доступа, основанную на понятиях **владельца**, **группы** и **остальных** пользователей. Права хранятся в специальном поле атрибутов файла.

Структура поля атрибутов файла

Поле атрибутов файла включает информацию о его типе и правах доступа. Права доступа кодируются в 9 битах, разбитых на три группы по 3 бита: * Права для **владельца** файла (User). * Права для **группы**, которой принадлежит файл (Group). * Права для **всех остальных** пользователей системы (Other).

Виды прав и их значение

В каждой из трех групп прав (`user`, `group`, `other`) могут быть установлены следующие права:

Право	Обозначение	Числовое значение (восьмеричное)	Значение для Файла	Значение для Каталога
Чтение	`r`	4	Возможность просматривать содержимое файла.	Возможность просматривать список файлов в каталоге.
Запись	`w`	2	Возможность изменять содержимое файла.	Возможность создавать, удалять и переименовывать файлы в каталоге.
Исполнение	`x`	1	Возможность выполнять файл как программу.	Возможность входить в каталог и получать доступ к его файлам/подкаталогам.

Права могут быть представлены в символьной форме (например, `rwxr-xr--`) или в числовой (восьмеричной) форме, где каждое число (от 0 до 7) является суммой значений установленных прав в соответствующей группе (например, `rwx` = 4+2+1=7, `r-x` = 4+0+1=5). Права для владельца, группы и остальных записываются последовательно (например, `755` соответствует `rwxr-xr-x`).

Особые признаки прав доступа

Существуют три особых признака, которые могут быть установлены для файлов или каталогов, влияющие на поведение при выполнении или доступе. Они также кодируются в поле атрибутов.

Признак	Символ (при наличии права `x`)	Символ (при отсутствии права `x`)	Числовое значение (восьмеричное, в начале)	Применение
Sticky-бит	`t` (в поле `other`)	`T` (в поле `other`)	1	Для каталогов: только владелец файла (или суперпользователь root) может удалять файлы из этого каталога, даже если у других есть право на запись в каталог.
SUID (Set User ID)	`s` (в поле `user`)	`S` (в поле `user`)	4	Для исполняемых файлов: позволяет любому пользователю запустить файл с правами владельца этого файла. **Потенциально опасно, использовать с осторожностью.**
SGID (Set Group ID)	`s` (в поле `group`)	`S` (в поле `group`)	2	Для исполняемых файлов: позволяет любому пользователю запустить файл с правами группы-владельца этого файла. Для каталогов: новые файлы, созданные в этом каталоге, будут иметь ту же группу, что и каталог. **Потенциально опасно.**

При установке SUID или SGID символ `x` в соответствующей группе прав заменяется на `s`. Если исполняемое право не было установлено, вместо `s` будет `S`. Sticky-бит заменяет `x` на `t` в группе "остальные". Если исполняемое право не было установлено, вместо `t` будет `T`. Числовое представление особых признаков ставится перед основными тремя цифрами прав (например, `4755` означает установленный SUID и права `755`).

Работа с файлами в языке Си

В языке программирования Си для работы с файлами и устройствами ввода/вывода используется абстракция **потоков данных (stream)**. Поток — это последовательность байтов, которая течет от источника к получателю. Внешние устройства (клавиатура, экран, принтер) и файлы на диске рассматриваются как потоки.

Потоки данных: Концепция

Поток данных в Си — это абстракция, позволяющая работать с различными источниками/приемниками данных (файлы, устройства) унифицированным способом. Библиотека стандартного ввода-вывода (stdio) управляет этими потоками, предоставляя буферизацию и преобразования данных для удобства программиста.

Пример (концептуальный):

```
#include <stdio.h>

int main() {
   int character;

   printf("Введите символ: "); // Вывод в поток stdout
   character = getchar();      // Чтение из потока stdin

   printf("Вы ввели: ");       // Вывод в поток stdout
   putchar(character);         // Вывод символа в поток stdout
   printf("\n");

   return 0;
}
```

В этом простом примере `getchar()` читает один символ из стандартного потока ввода (`stdin`), а `printf()` и `putchar()` записывают символы в стандартный поток вывода (`stdout`). С точки зрения программы, она просто работает с потоками байтов, не "зная", что `stdin` обычно привязан к клавиатуре, а `stdout` — к экрану.

Стандартные потоки

При запуске любой программы на Си автоматически открываются три стандартных потока, связанных с консолью:

- `stdin`: Стандартный поток ввода (обычно связан с клавиатурой).
- `stdout`: Стандартный поток вывода (обычно связан с экраном для обычного вывода).
- `stderr`: Стандартный поток ошибок (обычно связан с экраном для вывода сообщений об ошибках).

Поток `stderr` часто **небуферизован** для немедленного отображения ошибок. Эти потоки представлены указателями типа `FILE*` и объявлены в заголовочном файле `stdio.h`. Функции консольного ввода-вывода, такие как `scanf`, `printf`, `getchar`, `putchar`, являются высокоуровневыми функциями, работающими со `stdin` и `stdout`.

Структура `FILE`

Для управления потоками библиотека `stdio.h` использует структуру `FILE`. Указатель на эту структуру (`FILE *`) используется функциями для работы с файлами и потоками.

Структура `FILE` (часто реализуемая как `struct _IO_FILE`) содержит всю необходимую информацию о потоке: текущую позицию указателя чтения/записи, состояние буфера, флаги ошибок и конца файла. Программисты работают с файлами исключительно через указатели `FILE*` и функции стандартной библиотеки, не обращаясь напрямую к полям структуры `FILE`.

Общий алгоритм работы с файлом

Типичный цикл работы с файлом в Си выглядит так:

1. **Открыть поток** (`fopen`).
2. **Выполнить операции** (чтение/запись). При необходимости можно изменять текущую позицию в потоке (`fseek`, `rewind`).
3. **Закрыть поток** (`fclose`). **Обязательно!** Закрытие файла освобождает системные ресурсы и гарантирует сброс данных из буферов на диск.

Буферизация

Ввод-вывод в Си обычно буферизован. Это означает, что данные не передаются между программой и устройством напрямую при каждом вызове функции чтения/записи. Вместо этого данные сначала накапливаются в специальной области памяти — **буфере** — и только затем передаются на устройство или считываются с него блоками.

- **Как работает**: При записи данные сначала попадают в буфер вывода. Когда буфер заполняется, или принудительно (`fflush()`), или при закрытии потока, его содержимое записывается на устройство. При чтении блок данных считывается с устройства в буфер ввода, и затем программа читает данные из этого буфера по запросу.
- **Преимущество**: Буферизация значительно ускоряет операции ввода-вывода, уменьшая количество медленных обращений к устройствам хранения данных.
- **Принудительный сброс**: Функция `fflush(FILE *stream)` принудительно записывает содержимое буфера вывода на устройство. Размер буфера по умолчанию определяется константой `BUFSIZ` (обычно 512, 1024 или 4096 байт). Можно изменить поведение буферизации функциями `setbuf()` и `setvbuf()`. Для потоков ввода `fflush()` не определена стандартно, но может очищать буфер ввода в некоторых реализациях.

Режимы открытия файлов (функция `fopen`)

Режим	Описание (текстовый/бинарный)
`"r"`	Открыть текстовый файл для **чтения**. Файл должен существовать.
`"w"`	Открыть текстовый файл для **записи**. Создается новый файл или существующий очищается.
`"a"`	Открыть текстовый файл для **добавления** (записи в конец). Создается, если не существует.
`"r+"`	Открыть текстовый файл для **чтения и записи**. Файл должен существовать.
`"w+"`	Открыть текстовый файл для **чтения и записи**. Создается или очищается.
`"a+"`	Открыть текстовый файл для **чтения и добавления** (в конец). Создается, если не существует.
`"rb"`	Открыть **бинарный** файл для **чтения**.
`"wb"`	Открыть **бинарный** файл для **записи**.
`"ab"`	Открыть **bінарный** файл для **добавления**.
`"r+b"`	Открыть **бинарный** файл для **чтения и записи**.
`"w+b"`	Открыть **бинарный** файл для **чтения и записи**.
`"a+b"`	Открыть **бинарный** файл для **чтения и добавления**.

`FILE *fopen(const char *filename, const char *mode)`: Открывает файл с заданным именем (`filename`) в указанном режиме (`mode`) и возвращает указатель на `FILE` или `NULL` в случае ошибки. **Обязательно проверяйте возвращаемое значение на `NULL`!Пример открытия и закрытия с проверкой:**

```
#include <stdio.h>
#include <stdlib.h> // Для использования perror()

int main() {
    FILE *fp;
    char filename[] = "my_test_file.txt";

    // Попытка открыть файл для чтения
```

```
    if ((fp = fopen(filename, "r")) == NULL) {
        // Если fopen вернул NULL, значит, произошла ошибка (например, файл не найден)
        perror("Ошибка открытия файла для чтения"); // Выводит сообщение об ошибке из системы
        // return 1; // Завершить программу с кодом ошибки
        // Пример продолжения: попытка создать файл для записи
        printf("Попробуем создать файл для записи...\n");
        if ((fp = fopen(filename, "w")) == NULL) {
            perror("Ошибка создания файла для записи");
            return 1; // Завершить программу с кодом ошибки
        }
        printf("Файл успешно создан и открыт для записи.\n");
    } else {
        printf("Файл успешно открыт для чтения.\n");
    }

    // Здесь выполняются операции с файлом...

    // Закрытие файла
    if (fclose(fp) != 0) {
        // Если fclose вернул ненулевое значение, произошла ошибка при закрытии
        perror("Ошибка закрытия файла");
        return 1; // Завершить программу с кодом ошибки
    }
    printf("Файл успешно закрыт.\n");

    return 0; // Успешное завершение программы
}
```

Закрытие файла (функция `fclose`)

`int fclose(FILE *stream)`: Закрывает поток, связанный с указанным указателем `stream`. Возвращает 0 при успехе, `EOF` при ошибке. **Обязательно закрывайте открытые файлы** для освобождения системных ресурсов и гарантии записи буферизованных данных на диск.

Низкоуровневый файловый ввод-вывод (Файловые дескрипторы)

Параллельно с высокоуровневыми функциями `stdio.h`, существует низкоуровневый интерфейс работы с файлами, основанный на **системных вызовах** и **файловых дескрипторах**.

- **Файловый дескриптор (file descriptor, fd)** — это целое число (`int`), которое ядро операционной системы присваивает каждому открытому файлу или устройству для конкретного процесса. Дескрипторы уникальны в пределах одного процесса.
- Низкоуровневые функции (системные вызовы): `open()`, `read()`, `write()`, `close()`, `lseek()`. Они взаимодействуют напрямую с ядром ОС, минуя буферизацию, предоставляемую `stdio` (хотя ядро само может буферизовать).
- Стандартные дескрипторы (автоматически открыты): 0 (`stdin`), 1 (`stdout`), 2 (`stderr`). Функция `fileno(FILE *stream)` возвращает файловый дескриптор, связанный с потоком `FILE*`.

Текстовые и бинарные файлы

Файлы могут обрабатываться как **текстовые** или **бинарные** потоки. Режим открытия (`t` или `b` в строке режима `fopen`) определяет, какие преобразования данных будут выполняться библиотекой Си.

Текстовый поток

Последовательность символов, организованная в строки, разделенные символом новой строки (`\n`). При работе в текстовом режиме библиотека `stdio` может выполнять преобразования символов конца строки (например, `\n` в CR+LF в Windows). Чтение также может интерпретировать определенный символ (0x1A в Windows) как маркер конца файла (EOF).

Бинарный поток

Последовательность байтов без каких-либо преобразований. Данные читаются и записываются "как есть" (байт в байт). Используется для данных любых типов и позволяет прямой доступ по байтам.

Функции для работы с файлами в Си (`stdio.h`)

- **Основные функции:**

Функция	Назначение
`fopen`	Открывает файл.
`fclose`	Закрывает файл.
`fread`	Читает блоки данных из бинарного потока.
`fwrite`	Записывает блоки данных в бинарный поток.
`fgetc`	Читает один символ из потока.
`fputc`	Записывает один символ в поток.
`fscanf`	Читает форматированные данные из потока.
`fprintf`	Записывает форматированные данные в поток.
`fseek`	Устанавливает позицию указателя в потоке.
`rewind`	Устанавливает указатель в начало потока.
`feof`	Проверяет флаг конца файла.
`ferror`	Проверяет флаг ошибки потока.
`perror`	Выводит сообщение об ошибке в stderr.

- **Дополнительные функции:**

Функция	Назначение
`fgets`	Читает строку (до N символов или `\n`).
`fputs`	Записывает строку (без `\0`).
`fflush`	Сбрасывает буфер вывода.
`setbuf`	Настраивает буферизацию (откл или заданный буфер).
`setvbuf`	Более гибкая настройка буферизации.
`ftell`	Возвращает текущую позицию.
`clearerr`	Сбрасывает флаги ошибки/конца файла.
`fileno`	Возвращает файловый дескриптор (для низкоуровневых операций).

Пример текстового ввода/вывода:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   FILE *fp;
   int num;
   fp = fopen("data.txt", "w");
   if (fp == NULL) { perror("Write error"); return 1; }
   fprintf(fp, "%d", 123); fclose(fp);

   fp = fopen("data.txt", "r");
   if (fp == NULL) { perror("Read error"); return 1; }
   fscanf(fp, "%d", &num); fclose(fp);
   printf("Read number: %d\n", num);
   return 0;
}
```

Пример бинарного ввода/вывода:

```
#include <stdio.h>
#include <stdlib.h>

struct Data { int id; double value; };

int main() {
    FILE *fp;
    struct Data data_to_write = {1, 3.14};
    struct Data read_data;

    fp = fopen("binary.bin", "wb");
    if (fp == NULL) { perror("Write error"); return 1; }
    fwrite(&data_to_write, sizeof(struct Data), 1, fp);
    fclose(fp);

    fp = fopen("binary.bin", "rb");
    if (fp == NULL) { perror("Read error"); return 1; }
    fread(&read_data, sizeof(struct Data), 1, fp);
    fclose(fp);

    printf("Read binary data: id=%d, value=%f\n", read_data.id, read_data.value);
    return 0;
}
```

Позиционирование в файле (`fseek`, `rewind`, `ftell`)

- `rewind(FILE *stream)`: В начало файла.
- `fseek(FILE *stream, long int offset, int whence)`: Перемещает указатель. `offset` - смещение, `whence` - начало (`SEEK_SET`, `SEEK_CUR`, `SEEK_END`). Вернет 0 или ошибку.
- `ftell(FILE *stream)`: Текущая позиция (смещение от начала). Вернет -1L при ошибке.

Пример использования `fseek`:

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main() {
    FILE *fp;
    fp = fopen("seek_example.txt", "w+");
    if (fp == NULL) { perror("Ошибка открытия"); return 1; }
    fputs("abcdefghijklmnopqrstuvwxyz", fp); // Запись
    fseek(fp, 5, SEEK_SET); // На 5 байт от начала
    printf("Позиция (SEEK_SET + 5): %ld\n", ftell(fp)); // 5
    int c = fgetc(fp); // Читаем 'f'
    printf("Прочитан: %c\n", c); // f
    printf("Позиция (после чтения): %ld\n", ftell(fp)); // 6
    fseek(fp, -10, SEEK_END); // На 10 байт назад от конца
    printf("Позиция (SEEK_END - 10): %ld\n", ftell(fp)); // 16 (26-10)
    fclose(fp);
    return 0;
}
```

Проверка состояния потока: конец файла и ошибки

Проверяйте состояние после операций!

- `feof(FILE *stream)`: Проверяет флаг конца файла (после неудачной попытки чтения). **Не использовать как единственное условие цикла чтения**, может привести к ошибкам. Лучше проверять результат самой функции чтения.
- `ferror(FILE *stream)`: Проверяет флаг ошибки потока (устанавливается при ошибке, сбрасывается `clearerr`).
- `perror(const char *str)`: Выводит сообщение об ошибке в `stderr`. `str` + системное сообщение по `errno`.

Пример использования `feof` и `ferror`:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;
    int c;
    fp = fopen("example_read.txt", "r");
    if (fp == NULL) {
        perror("Ошибка открытия example_read.txt"); // Используем perror при ошибке открытия
```

```
        return 1;
    }

    printf("Читаем файл:\n");
    while ((c = fgetc(fp)) != EOF) { // Корректный цикл: проверка результата fgetc
        putchar(c);
    }

    // После цикла проверяем, почему вышли
    if (feof(fp)) {
        printf("\nДостигнут конец файла (feof).\n");
    } else if (ferror(fp)) {
        perror("Ошибка чтения из файла (ferror)"); // Используем perror при ошибке чтения
    }

    fclose(fp);
    return 0;
}
```

Пример использования `perror` при ошибке открытия:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;
    // Попытка открыть файл, который не существует - errno установится
    fp = fopen("non_existent_directory/non_existent_file.txt", "r");

    if (fp == NULL) {
        // perror прочитает errno и выведет системное сообщение
        perror("Ошибка открытия файла"); // "Ошибка открытия файла" - наш префикс
        return 1;
    }

    // Если открыт успешно, работаем с файлом...

    fclose(fp);
    return 0;

        }
```